

1. Instalación y Preparación

Si aún no tienes Conda, te recomiendo instalar **Miniconda** para Apple Silicon (M1/M2/M3/M4) para mantener tu sistema ligero o para windows.

1. **Crea el entorno virtual con Conda:** Vamos a crear un entorno llamado `fastapi-env` especificando una versión de Python reciente.

Bash

```
conda create -n fastapi-env python=3.11
```

2. **Activa el entorno:**

Bash

```
conda activate fastapi-env
```

3. **Instala las dependencias:** En Conda puedes usar tanto `conda install` como `pip`. Para FastAPI, lo más común y actualizado suele ser `pip` dentro del entorno de Conda:

Bash

```
pip install fastapi uvicorn
```

2. Estructura de Microservicios: De la Idea al Código

Cuando hablamos de microservicios, no solo lanzamos un script; organizamos el código para que sea escalable. Imagina que estás creando un microservicio para gestionar **sensores** o **usuarios**.

Estructura de archivos recomendada:

Plaintext

```
mi_proyecto/
├── app/
│   ├── __init__.py
│   ├── main.py          <-- Punto de entrada
│   ├── schemas.py       <-- Modelos de datos (Pydantic)
│   ├── database.py       <-- Conexión a datos
└── environment.yml       <-- Configuración de Conda
```

Ejemplo Pro: Validación de Datos

En un microservicio, la integridad es clave. Usaremos **Pydantic** para asegurar que los datos que recibimos son correctos:

Python

```
# app/schemas.py
from pydantic import BaseModel, Field

class SensorData(BaseModel):
    sensor_id: str
    valor: float = Field(..., gt=-50, lt=100) # Validación: entre -50
    y 100
    unidad: str = "Celsius"
```

El Servidor (Punto de entrada)

Python

```
# app/main.py
from fastapi import FastAPI
from .schemas import SensorData

app = FastAPI(title="Microservicio de Monitoreo")

@app.post("/telemetry")
async def recibir_datos(data: SensorData):
    return {"status": "recibido", "sensor": data.sensor_id, "lectura":
    data.valor}
```

3. Ventaja de Conda: El archivo `environment.yml`

Para que tu microservicio sea reproducible (que funcione igual en tu Mac M4 que en un servidor Linux), Conda usa archivos YAML. Puedes exportar tu entorno actual con:

Bash

```
conda env export > environment.yml
```

4. Por qué Conda brilla en microservicios con FastAPI

- **Aislamiento Total:** Si necesitas una versión específica de una librería de C++ o herramientas de compilación para ARM64, Conda las maneja mejor que `venv`.
- **Facilidad para IA:** Si tu microservicio de FastAPI va a procesar imágenes o usar modelos de lenguaje, instalar `numpy`, `pandas` o `scikit-learn` vía Conda suele evitar errores de compatibilidad con el chip M4.
- **Limpieza:** Con un simple `conda env remove -n fastapi-env` borras todo rastro del proyecto sin ensuciar tu sistema operativo.

Ejecución final:

Recuerda que con la estructura de carpetas, el comando cambia ligeramente:

Bash

```
uvicorn app.main:app --reload
```

Meter Todo a un Docker

